

Curso Inicial Front-End

Clase 12:

Fundamentos de

Programación con JS

Módulo 2: Dominio de JavaScript y Lógica de Programación



Presentación

¡Bienvenido al motor de la web! En esta clase daremos nuestros primeros pasos en **JavaScript**, el lenguaje que permite que las páginas web dejen de ser simples documentos y se conviertan en aplicaciones vivas. Entenderemos qué sucede "bajo el capó" con el motor V8, instalaremos **Node.js** y dominaremos el uso profesional de las variables, la base de toda la lógica de programación.



Objetivos

- Comprender el ecosistema de **JavaScript**: Motor V8, **Node.js** y su rol en la industria.
- Dominar el ciclo de vida de una variable: Inicialización, Asignación e Invocación.
- Diferenciar técnicamente entre tipos de datos por **Valor** y por **Referencia**.



Bloques Temáticos

- **Tema 1: Tipos de datos primitivos, variables (let/const) y operadores.**
 - ¿Qué es [JavaScript](#)? El lenguaje de cabecera de la diplomatura.
 - Historia y Por qué Web: De un script de 10 días a dominar el mundo.
 - El Motor V8 y [Node.js](#): Instalación y comprobación.
 - Variables: Momentos de vida. Inicialización, Asignación e Invocación.
 - Tipos de Memoria: Valor vs. Referencia.
 - Operadores de Asignación Compuesta.

Hasta ahora hemos construido la estructura de nuestra web con [HTML](#) y la hemos pintado con [CSS](#). Sin embargo, nuestra página es solo un póster digital estático; no puede hacer cálculos, no puede reaccionar inteligentemente si un usuario escribe mal su contraseña, ni puede guardar datos.

Hoy entraremos al mundo de la lógica. Aprenderemos [JavaScript \(JS\)](#), el cerebro de la web, que nos permitirá darle interactividad real a nuestros proyectos.

1. ¿Qué es JavaScript y por qué es el Rey?

JavaScript nació en 1995 (creado en solo 10 días por Brendan Eich). Su éxito se debe a que es el **único lenguaje que los navegadores entienden nativamente**. A lo largo de este curso, **JS** será nuestro **lenguaje de cabecera**: lo usaremos para el Frontend (**React**) y para el Backend (**Node.js**).

Características Técnicas: - **Tipado Dinámico:** No necesitas decirle que una variable es un número; él lo deduce solo. - **Multiparadigma:** Permite programar de forma Funcional u Orientada a Objetos (POO). - **Interpretado:** No necesita compilarse para ejecutarse.

I El Motor V8 y **Node.js**

Para ejecutar **JS** fuera del navegador, necesitamos **Node.js**, que utiliza el motor **V8** (el mismo de Google Chrome). 1. **Descarga:** Ve a nodejs.org e instala la versión **LTS**. 2. **Prueba de Instalación:** Abre una terminal y escribe: `node -v` Si aparece un número de versión (ej: v20.x.x), ¡felicitaciones! Tu computadora ya habla **JavaScript**.

2. Los Momentos de una Variable

Para entender las diferencias entre `var`, `let` y `const`, debemos entender los 3 momentos: 1. **Inicialización (Declaración):** Cuando creas el espacio en memoria. `let nombre;` 2. **Asignación:** Cuando le das un valor. `nombre = "Juan";` 3. **Invocación:** Cuando usas la variable. `console.log(nombre);`

Variable	¿Se puede re-declarar?	¿Se puede re-asignar?	Hoisting (Elevar)
const	✗ No	✗ No	✗ No
let	✗ No	✓ Sí	✗ No
var	✓ Sí (Peligroso)	✓ Sí	✓ Sí (Causa errores)

3. Valor vs. Referencia: La Trampa de la Memoria

- **Por Valor (Primitivos):** Si copias un número a otra variable, se crea una copia independiente. Si cambias una, la otra no cambia.
- **Por Referencia (Objetos/Arrays):** Si copias un objeto a otra variable, ambas apuntan al mismo lugar. Si cambias una, ¡la otra también cambia!

4. Operadores de Asignación Compuesta

Ahorra código usando atajos para cálculos: - `x += 5` es igual a `x = x + 5` - `x -= 5` es igual a `x = x - 5` - `x *= 2` es igual a `x = x * 2` - `x /= 2` es igual a `x = x / 2`

5. Reglas de Oro

En programación necesitamos recordar datos (como la vida de un jugador o su nombre). Para eso usamos **Variables**.

Imagina que una variable es una cajita de cartón a la que le pegas una etiqueta con un nombre. Técnicamente, es un espacio reservado en la Memoria RAM de tu computadora al que le asignamos un nombre fácil de recordar para guardar un dato y poder reutilizarlo más tarde.

En lenguaje simple: Una variable es como una etiqueta que le pones a una caja para encontrar tus cosas más rápido. El Stack es tu bolsillo (cosas a mano) y el Heap es un depósito grande para las cosas pesadas.

Traducción Técnica: Las variables son identificadores simbólicos para ubicaciones en la memoria física. Al declarar una variable, el motor de **JavaScript** reserva un espacio en el **Stack** (para tipos primitivos) o en el **Heap** (para objetos). La palabra clave **const** crea una referencia de solo lectura (binding inmutable), mientras que **let** permite la re-asignación en el mismo ámbito léxico.

Hoy en día usamos dos "tipos de cajas":

- **const (Constante):** Es una caja fuerte. Una vez que guardas el dato y la cierras, **el dato no puede mutar ni cambiarse**. Se usa para cosas que nunca cambian en la vida de la página (ej: tu fecha de nacimiento).
- **let (Variable elástica):** Es una caja normal. Puedes abrirla, tirar el dato viejo a la basura y guardar un dato nuevo cuantas veces quieras (ej: el puntaje de un juego que sube o baja).

```
// La palabra clave 'let' crea la caja, 'edad' es el nombre, el '=' guarda el dato 20 adentro.  
let edad = 20;  
edad = 21; // ✅ Válido. Abrimos la caja y reemplazamos el 20 por un 21.  
  
const documentoIdentidad = 35111222;  
documentoIdentidad = 35111333; // ❌ ERROR CRÍTICO. El navegador explotará.
```

Comparativa de Variables en Entrevistas Laborales

Declaración	¿Se puede reasignar?	¿Tiene Hoisting (Se eleva al inicio)?	Recomendación Industrial
<code>var</code>	Sí	Sí. Se eleva y toma el valor <code>undefined</code> .	✗ NUNCA lo uses. Es la forma vieja de JS (pre-2015) y causa bugs horribles.
<code>let</code>	Sí	Sí, pero se bloquea por seguridad (TDZ).	⚠ Úsalo solo si sabes matemáticamente que ese valor cambiará a futuro.
<code>const</code>	No	Sí, pero se bloquea por seguridad (TDZ).	✓ Úsalo por defecto SIEMPRE. Un código con muchas constantes es seguro.

6. Tipado Dinámico: Una caja que cambia de forma

La "forma" de los datos que guardas en las cajas se conoce como **Tipo de Dato**. A diferencia de lenguajes antiguos y rígidos (como C++), **JavaScript** es de **Tipado Dinámico**. Esto significa que una caja no se casa con un tipo de dato. Una variable que hoy guarda un número `let x = 10;`, en la línea siguiente puede mutar y guardar una palabra `x = "Hola";`.

- Para saber qué tipo de dato hay dentro de una caja en cualquier momento, le preguntamos a **JS** usando la palabra reservada **typeof**: `javascript let misterio = 50; console.log(typeof misterio);` // En la consola aparecerá la palabra "number"

7. Tipos de Datos (Los Átomos de la Programación)

JavaScript clasifica la información del mundo real en diferentes categorías técnicas:

Tipos Primitivos (Datos simples de un solo valor):

- **Strings (Cadenas de texto):** Palabras o letras. Para que **JS** sepa que es un texto y no una variable, **siempre** deben ir envueltas entre comillas simples (') o dobles ("). Ej: "Juan".
- **Numbers (Números):** Sirven para hacer matemática. Se escriben "desnudos", sin comillas. Ej: 25 o 9.99.
 - *La Trampa NaN (Not a Number):* Ocurre cuando el navegador intenta hacer matemática con algo absurdo (ej: "manzana" / 2). Significa que la operación matemática fracasó.
- **Booleans (Boleanos):** Son interruptores lógicos. Solo pueden tener dos estados exactos: **true** (verdadero/encendido) o **false** (falso/apagado). Se escriben sin comillas.
- **El Vacío: undefined vs null**
 - **undefined:** Significa que la variable existe, pero **nadie nunca le guardó un dato**. Es una ausencia accidental.
 - **null:** Significa que la caja fue vaciada **a propósito** por el programador. Es una ausencia intencional.

8. Template Literals (Cadenas de Plantilla Modernas)

Cuando queremos mezclar un texto fijo con el valor dinámico de una variable, ya no usamos el signo `+` (concatenación antigua), porque es propenso a errores de espacios. El estándar moderno es usar **Backticks** (comillas invertidas ```) y la sintaxis de inyección `${ }`:

```
const nombre = "Ana";
const edad = 30;

// ❌ Forma vieja (Incómoda):
// console.log("Hola, me llamo " + nombre + " y tengo " + edad + " años.");

// ✅ Forma moderna (Template Literals):
console.log(`Hola, me llamo ${nombre} y tengo ${edad} años.`);
```

9. Operadores Aritméticos Básicos

Tu computadora, en el fondo, es una calculadora glorificada. **JS** usa los símbolos matemáticos estándar:

```
let suma = 5 + 5;           // El resultado guardado es 10
let resta = 20 - 5;         // El resultado guardado es 15
let multiplicacion = 2 * 3; // Usa el asterisco. Resultado: 6
let division = 10 / 2;      // Usa la barra. Resultado: 5
let resto = 10 % 3;         // Se llama 'Módulo'. Devuelve lo que sobra de la división (1)
```

Documentación Oficial (Referencias)

- **CSS:** <https://developer.mozilla.org/es/docs/Web/CSS>
- **HTML:** <https://developer.mozilla.org/es/docs/Web/HTML>
- **JS:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
- **JavaScript:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
- **Node.js:** <https://nodejs.org/es/docs/>
- **React:** <https://es.react.dev/>